

```

import { q, one } from '../db.js';

// The only place in the platform that reads across businesses, and it can only
// see rows a business explicitly published. Everything else stays tenant-scoped.
//
// Booking and payment are not moved. The source keeps those. What moves is the
// looking: one search instead of nine phone calls.
export async function search({ category, area = null, date = null, partySize = 1,
  const params = [category, partySize];
  const where = [
    `l.published = true`,
    `l.category = $1`,
    `l.units_left >= $2`,
  ];
  if (date) { params.push(date); where.push(`l.on_date = ${params.length}`); }
  if (area) { params.push(area); where.push(`l.area = ${params.length}`); }
  if (maxPrice != null) { params.push(maxPrice); where.push(`l.price <= ${params

const rows = await q(
  `select l.*, b.slug, b.display_name
    from listing l join business b on b.id = l.business_id
   where ${where.join(' and ')}
   order by l.price nulls last, l.starts`,
  params
);

const businesses = new Set(rows.map(r => r.business_id));

await one(
  `insert into directory_search (category, area, on_date, party_size, matched,
    values ($1,$2,$3,$4,$5,$6) returning *`,
  [category, area, date, partySize, rows.length, businesses.size]
);

return {
  category, date, partySize,
  matched: rows.length,
  businesses: businesses.size,
  results: rows.map(r => ({
    listingId: r.id,
    business: r.display_name,
    slug: r.slug,
    title: r.title,
    area: r.area,

```

```

        date: iso(r.on_date),
        starts: r.starts,
        ends: r.ends,
        unitsLeft: r.units_left,
        price: r.price,
        // Where it actually lives. The visitor sees one shelf; the booking
        // still happens wherever the business already sells.
        source: r.source,
        bookUrl: r.book_url,
    ))),
};
}

// Clicking through to the source is recorded, so the business sees the demand
// even though the transaction happens on someone else's platform.
export async function recordReferral({ listingId, partySize = 1 }) {
    const listing = await one(`select * from listing where id = $1`, [listingId]);
    if (!listing) return null;
    return one(
        `insert into referral_click (business_id, listing_id, source, on_date, party_
        values ($1,$2,$3,$4,$5) returning *`,
        [listing.business_id, listing.id, listing.source, listing.on_date, partySize]
    );
}

// What the shelf looked like, in aggregate. Unmatched demand is the useful half.
export async function demand({ category = null, sinceDays = 30 } = {}) {
    const rows = await q(
        `select category, on_date, party_size, matched, businesses_matched, created_a
        from directory_search
        where ($1::text is null or category = $1)
        order by created_at desc limit 200`,
        [category]
    );
    return {
        searches: rows.length,
        unmatched: rows.filter(r => r.matched === 0).length,
        rows: rows.map(r => ({ ...r, on_date: iso(r.on_date) })),
    };
}

function iso(v) {
    return v instanceof Date ? v.toISOString().slice(0, 10) : String(v ?? '').slice
}

```